



ASTRALIS UI

The Work Report

What P0, P1 and P2 actually delivered — every fix and every new component explained in plain words — and where Astralis UI stands today.

8 July 2026

Companion: The Road Ahead.

Plain-language edition — concepts explained inline

1. The short story

We ran a full audit of the library, found everything that was broken or missing, and then fixed all of it in three phases:

- **P0** — things that were silently broken. Six bugs that made real features do nothing, with no error anywhere.
- **P1** — things that were inconsistent or incomplete. Theming gaps, accessibility gaps, messy packaging.
- **P2** — things that were missing. Twelve new components, modern React patterns, and an AI layer.

All of it is committed in eight commits, from `db4f6c8` to `f002fe1`. Every change was type-checked, built, and tested in a real browser before it was committed.

Astralis UI now has 62 components across 9 categories. Every one of them has a docs page with live demos.

2. What P0 fixed — the silent breakage

These six bugs shared one nasty trait: nothing crashed. Features just quietly did nothing.

Concept — why CSS can fail silently: a browser that meets a class name it doesn't know simply ignores it. No error, no warning. Our library pre-compiles all its CSS ahead of time, so if a class was never generated at build time, the component "works" but is missing a style. That's why every Po bug here was invisible until we went looking.

Concept — design tokens: a token is a named value, like `--duration-fast: 150ms` or `--color-brand-500`. Components never hard-code values; they reference tokens. Change the token, and everything using it changes. Our tokens live in CSS files under `theme/tokens/`.

1. Animation speeds were dead. We named our animation speed tokens `--duration-fast`, `--duration-moderate` and so on. Tailwind only reads that kind of token from a namespace called `--transition-duration-*`. Wrong name, so no CSS was ever generated — every animation ran at a default speed instead of the designed one. One rename in `tailwind-entry.css` fixed the whole library at once.

2. 87 sizing props did nothing. Props like `boxSize="md"` or `maxW="8xl"` pointed at CSS classes that were never generated. A user setting them got nothing, with no warning. We revived most of them by adding the missing token definitions, deleted the handful that were genuinely invalid CSS, and — most importantly — **built a guard so this can never happen again** (see section 5).

Concept — the safelist: Tailwind only generates CSS for classes it can see in the source code. Our responsive props build class names at runtime (`"astralis:md:p-4"`), which Tailwind can't see. So a script lists ("safelists") every class the engine could ever build, and forces Tailwind to generate them ahead of time. If that list has a hole, a responsive style silently doesn't exist.

3. Responsive props half-worked. The script that pre-generates responsive classes (`gen-responsive-safelist.mjs`) only saw the *first* class in any multi-class value. Example of the damage: a Separator set to be horizontal on mobile and vertical on desktop simply **disappeared** on desktop — its border class was never generated. Fixed the script, and also fixed Separator and Float so each orientation properly cancels the other one's styles.

Concept — two layers of tokens: raw shades (`brand-500` = a specific violet) and roles (`brand-solid` = "the fill color of a solid button", which points at a shade). Components paint with roles, so a theme can re-point the roles without touching any component. The bug: CSS resolves a role's pointer once, where the role is declared — so overriding the shade lower down changed nothing. The fix re-declares the roles themselves.

4. The brand color API didn't recolor components. `<AstralisProvider tokens={{ brandColor: "#8b5cf6" }}>` is supposed to repaint the whole library. It only changed the raw color shades — the "role" tokens components actually paint with were locked in at page level, so almost everything stayed yellow. The

provider (`provider.tsx`) now re-declares the role tokens too, per light/dark theme, and picks a readable text color automatically.

5. Stacked overlays broke each other. Open a Modal, open a Drawer on top, close the Drawer — and page scrolling came back while the Modal was still open. Also, one press of Escape closed *every* open overlay at once. We added a small shared "overlay stack" (`overlay-stack.ts`): scroll stays locked until the *last* overlay closes, and Escape peels one layer at a time, top first.

6. Select and MultiSelect were invisible to forms. Their values never reached a normal `<form>` submit, because there was no real input under the hood. Both now take a `name` prop and render hidden inputs, so `FormData` sees them like any native field.

3. What P1 fixed — coherence and depth

Theming coherence. Tabs had no `colorScheme` prop at all — it was hard-wired to the brand color while every similar component was themeable. It now rides the accent channel like everything else. Carousel's half-themed controls were unified, and the Switch thumb's raw `bg-white` got a documented reason to exist.

Accessibility depth. The library-wide gaps from the audit are closed:

- `prefers-reduced-motion` support — Marquee pauses, Carousel autoplay holds (new hook: `use-prefers-reduced-motion.ts`).
- Every form control now announces its help and error text: Field hands out ids, and inputs wire them into `aria-describedby` automatically. (Concept: `aria-describedby` is how a screen reader knows that "Must be 8+ characters" belongs to the password box. Without the link, the helper text exists visually but is never read out with the field.)
- Calendar got full arrow-key navigation (arrows move a day, PageUp/Down a month, Home/End to week edges).
- Group labels for Radio/Checkbox groups, per-cell labels for PinInput ("digit 1 of 4"), labels on Carousel arrows and dropdown search boxes.

API consistency. Tag and Badge now share the same size scale, `Highlight` no longer crashes on non-text children, and components like Badge and the Modal/Popover panels now pass through extra props (like `data-*` attributes) instead of dropping them.

Packaging. The published shape was rebuilt: one file per module instead of a single 457 KB bundle, `"use client"` markers only where needed, a `sideEffects` flag, and a corrected exports map. Importing one Button no longer means shipping the whole library.

Concept — tree-shaking: a bundler drops code the app never imports — but only if it can prove the code is safe to drop. One giant file can't be split, and without a `"sideEffects": false` promise in `package.json` the bundler won't even try. Both were blocking it; both are fixed.

4. What P2 added — growth

Twelve new components

Category	New components	The interesting part
Feedback (new category)	Alert, Progress, Skeleton, Spinner, Toast	The audit found we had <i>no</i> feedback components while every competitor ships all five. Toast is the big one: call <code>toast.success("Saved")</code> from anywhere — no setup beyond one <code><Toaster /></code> (toast-store.ts). Its timers pause while you hover and resume with the time that was left.
Menu (new category)	Menu	A real dropdown action menu: arrow keys move (skipping disabled items), typing jumps to a match, Enter activates, Escape returns focus to the button. Built on a new reusable hook, <code>use-roving-focus.ts</code> . (Concept: "roving focus" means only one item in a widget is tabbable; arrow keys move a single roaming focus between items. It's how every native menu behaves, and the accessibility standard — the APG — requires it.)
Data entry	Textarea, NumberInput, Combobox	Combobox is a search-as-you-type picker. Building it forced Select, MultiSelect and Combobox onto one shared options engine (<code>shared/options.tsx</code>) so the three can never drift apart.
Typography / Navigation	Kbd, Link, Breadcrumb	Small but expected everywhere. Breadcrumb inserts its separators automatically — you can't get them wrong.

Modernization

React 19 style. 43 components no longer use the old `forwardRef` wrapper — `ref` is now a plain prop. The complicated generic components (Box, Text, ...) stay as they are on purpose; converting them buys nothing yet.

Concept — refs and forwardRef: a ref is how a parent gets its hands on the real DOM element a component renders (to focus it, measure it...). Old React made components opt in through a wrapper called `forwardRef`, which added noise and broke TypeScript on our generic components. React 19 delivers `ref` like any other prop, so the wrapper — and its type-system pain — can go.

Better brand colors. The math that turns one brand color into ten shades was rewritten from simple color mixing to **OKLCH**.

Concept — OKLCH: a way of describing a color by three dials that match human eyesight: **L**ightness (dark↔light), **C**hroma (grey↔vivid), **H**ue (which color). To make a darker shade you turn only the L dial — the color stays itself, just darker. Mixing with black (the old way) accidentally turns all three dials, which is why saturated colors went muddy. Tailwind's own built-in palette uses OKLCH for exactly this reason.

The practical difference: a violet brand used to fade into grey-brown mud at the dark end; now violet-900 is a deep, rich violet. The math lives next to the provider fix in `provider.tsx` — about sixty dependency-free lines.

The AI layer

Coding assistants don't know Astralis exists — it's too new to be in their training data. So we made the docs machine-readable:

Concept — *llms.txt* and MCP: `llms.txt` is a convention (like `robots.txt`, but for AI): a plain-text index at the site root that tells an AI agent what documentation exists and where. **MCP** (Model Context Protocol) goes further — it's a standard plug that lets tools like Claude Code call functions we define ("list the components", "give me Button's props"). Both mean an assistant fetches current, correct answers about Astralis instead of guessing.

- `/llms.txt` — an index AI agents read to discover every component.
- `/docs/components/button.md` (and every other page) — the full docs as plain markdown, demo source code included (`docs-markdown.ts`).
- **An MCP server** (`packages/astralis-mcp/server.mjs`) — tools like Claude Code can connect to it and ask for component lists, props, examples, and theming rules. It reads the docs straight from disk; no build, no hosting needed.

5. The guard that keeps it fixed

The scariest PO bugs were the *silent* ones. The permanent answer is a check that runs inside every CSS build ([check-css-coverage.mjs](#)): it extracts every class the components can possibly emit and asserts each one actually exists in the compiled stylesheet — responsive variants included. If a future change breaks a class, **the build fails loudly** instead of a component failing quietly.

Right now it verifies 3,040 base classes plus 2,859 responsive ones at four breakpoints each — all green.

6. Where the library stands

The numbers.

Components	62, in 9 categories (Buttons, Layout, Typography, Data Entry, Disclosure, Navigation, Menu, Overlay, Data Display, Feedback)
Docs	62 pages, 130+ live demos, sidebar + search-ready structure
Compiled CSS	1.0 MB raw / ~118 KB gzipped, every class verified
Dependencies at runtime	Four small ones (cva, clsx, tailwind-merge, qrcode) — no Radix, no Floating UI, no framer-motion
React	19-first, <code>ref</code> as a plain prop in new code

Against the competition. The audit's harshest findings are now closed: the missing Feedback category (was the #1 gap — Toast ships in 11 of 11 libraries we surveyed), the missing Menu (#2), Combobox, native form support, and the AI/agent surface (llms.txt + MCP) that only the biggest libraries currently offer. The architecture bets — precompiled CSS, semantic tokens, runtime brand theming — were validated by the market moving the same way.

What kind of library this is now. A styled, batteries-included React 19 library that owns its own behavior (five small hooks instead of headless dependencies), hides its Tailwind internals behind a prefix, recolors itself at runtime from a single brand color, and can prove — on every build — that every class it emits really exists.

The companion document, *The Road Ahead*, covers what we deliberately left for later and the recommended order to tackle it.